

Final examination on Logic Circuits Design (Group 1)

Christian Rinderknecht

12 December 2007

1 Addition of negabinary numbers

Question. The *negabinary* representation of a number is similar to the binary representation, except that the base is not 2 but -2 . Therefore, the general shape of the negabinary representation with n bits $b_{n-1}, b_{n-2}, \dots, b_0$ is

$$b_{n-1}(-2)^{n-1} + b_{n-2}(-2)^{n-2} + \dots + b_1(-2)^1 + b_0$$

Devise a C function adding two numbers expressed in the negabinary system:

```
int* addNeg2 (int* a, int* b);
```

where a number is represented as a pointer to an array of integers. For example, the implementation of the negabinary 1101_{-2} is a pointer to the leftmost cell of the array

4	1	1	0	1
---	---	---	---	---

Hints. Given two negabinary numbers $b_{n-1}b_{n-2} \dots b_0$ and $b'_{n-1}b'_{n-2} \dots b'_0$,

1. find all the possible carries by considering $b_0 + b'_0$, then $b_i + b'_i + \text{carry}$;
2. make a table associating each possible values of $b_i + b'_i + \text{carry}$ to the next carry and the correct bit;
3. in order to find the maximum size of the output, consider what happens when a nonzero carry comes out of the position $n - 1$;
4. compute the size of the output with your algorithm with one carry (initially 0) and one bit, incrementing a variable `size` each time a position is done, then check the last nonzero carry and fix `size` accordingly;
5. allocate the result array with length `size + 1`;
6. rerun the algorithm (a loop) but, this time, fix the bits and save them in the resulting array.